
1. Symmetric Encryption

Definition:

Uses **one single key** for both **encryption** and **decryption**.

Key Point:

- Same key must be **shared secretly** between sender and receiver.

Algorithms:

- AES (Advanced Encryption Standard)
- DES, 3DES
- Blowfish

Example:

Encrypt: PlainText + Key → CipherText

Decrypt: CipherText + Same Key → PlainText

Pros:

- Very **fast**
- Good for **large data** encryption

Cons:

- **Key distribution** is a challenge
- If key is compromised, all data is compromised

Use Case:

- Encrypting data at rest (e.g., AWS EBS, RDS encryption)
 - File or volume encryption
-

2. Asymmetric Encryption

Definition:

Uses a **pair of keys**:

- **Public key** (used to encrypt)
- **Private key** (used to decrypt)

Key Point:

- **Public key** can be shared openly
- **Private key** must be kept secure

Algorithms:

- RSA
- ECC (Elliptic Curve Cryptography)

Example:

Encrypt: PlainText + Public Key → CipherText

Decrypt: CipherText + Private Key → PlainText

Pros:

- Solves the **key distribution problem**
- Enables **digital signatures**

✗ Cons:

- **Slower** than symmetric
- Not ideal for large data

📖 Use Case:

- SSL/TLS (e.g., HTTPS)
 - Secure login systems
 - Key exchange (e.g., SSH, PGP)
-

📄 3. Digital Certificates

📖 Definition:

A **digital document** that uses **asymmetric encryption** to associate a **public key** with an **entity** (e.g., a website, user, or device).

🔑 Contains:

- Public Key
- Identity Info (domain name, org, etc.)
- Certificate Authority (CA) signature

📦 Example:

Certificate = [Public Key + Website Info] + Signed by CA

✅ Pros:

- Verifies **authenticity** of sender

- Ensures **secure communication** (via SSL/TLS)

✗ Cons:

- Depends on **trust in Certificate Authorities (CA)**



Use Case:

- HTTPS websites (SSL Certificates)
- Code signing
- Email encryption (S/MIME)



Relationship:

Scenario	Symmetric	Asymmetric	Certificates
Encrypting large data	✓ Preferred	✗ Not efficient	✗ Not used directly
Secure Key Exchange	✗ Not possible	✓ Used	✓ Provides trust
Identity Verification	✗	✓ With signature	✓ Via CA
HTTPS Website Communication	✓ (AES)	✓ (RSA/ECDSA)	✓ (SSL Certificate)



Real-World Example: HTTPS

1. Browser gets a **certificate** from a website.
2. Certificate has the website's **public key**, signed by a **trusted CA**.
3. Browser verifies the **CA signature**.

4. Browser generates a **symmetric key** (e.g., AES), encrypts it with the **public key** from the certificate.
 5. Server decrypts the key using its **private key**.
 6. All future communication uses **fast symmetric encryption**.
-